CS 61A — Week 2

Solutions to the handout problems (code for the exercises is in `solutions/`)

P1.

1. `0 or 5 and 3` is `0 or (5 and 3)`. `0` is false, so evaluate `5 and 3`: `5` is true, so the value is `3`. Displays `3`.
2. `not 0 and None` is `(not 0) and None`. `not 0` is `True`, so the value is that of the right side, `None`. Nothing is displayed.
3. `1 / 0 or 2`: the left side is evaluated first and is an `ERROR`.
4. `2 or 1 / 0`: `2` is true, evaluation stops. Displays `2`; the division never happens.
5. `print(3) or print(4) and 5` is `print(3) or (print(4) and 5)`. `print(3)` displays `3`, value `None` (false), so continue. `print(4)` displays `4`, value `None` (false), so the `and` stops with `None`. The whole value is `None`: nothing more is displayed.
6. `'' or 0`: both false; the value is the last one, `0`. Displays `0`.

P2. As written: `5 > 3`, so `a` is displayed and `x` becomes `2`. The second `if` is a separate statement: `2 > 1`, so `b` is displayed. Output: `a`, `b`. With `elif`, the three clauses are one statement and only the first true clause runs: output `a` only.

P3. `n` takes the values 37, 18, 9, 4, 2, 1, then 0. The odd ones are printed as they occur: `37`, `9`, `1`. Six passes were made, so the last line is `6`.

P4. Compare the last digit with the one before it; stop at the first pair with equal parity. The loop needs at least two digits, so the condition is `n >= 10`.

```
def alternates(n):
    while n >= 10:
        if n % 2 == n // 10 % 2:
            return False
        n = n // 10
    return True
```

`n % 2` is the parity of the last digit (the parity of a number is the parity of its last digit), and `n // 10 % 2` is the parity of the digit before it. The `return False` inside the loop ends the function at the first failure; `return True` is reached only if no pair failed.

P5.

1. `h = twice(inc)`: `h` is the function `g` whose `f` is `inc`.
2. `h(0)` is `inc(inc(0))`: inner call displays `inc 0`, returns `1`; outer displays `inc 1`, returns `2`. `print` displays `2`.
3. `twice(twice(inc))` is a function that applies `twice(inc)` twice, so it applies `inc` four times. On `10`: displays `inc 10`, `inc 11`, `inc 12`, `inc 13`; value `14`.

Displayed: `inc 0`, `inc 1`, `2`, `inc 10`, `inc 11`, `inc 12`, `inc 13`, `14`.

P6.

1. Global: `x` is `10`; `outer` is `func outer(x) [parent=Global]`.
2. `outer(1)`: frame `f1: outer [parent=Global]`, `x` is `1`. The inner `def` creates `func inner(y) [parent=f1]`. Then `x = x * 2` rebinds `x` in `f1` to `2`. Returns the `inner` function; global `f` is bound to it.
3. `x = 100` rebinds the *global* `x`.
4. `f(5)`: frame `f2: inner [parent=f1]`, `y` is `5`. `x` is not in `f2`; it is found in `f1`, where it is now `2`. Returns `7`.

Displayed: `7`. The global `x` (`100`) is never reached, and the value of `x` at the time `inner` was defined (`1`) is not remembered: lookup happens when the body runs.

E1 (published answer: three lines). Name the functions. `og` is the built-in print. The first `new_print(print)` returns a function, call it `f_A`, whose `print` (found in its parent frame) is the built-in. The second `new_print(print)` is called with `f_A` as its argument and returns `f_B`, whose `print` is `f_A`. The global name `print` is now `f_B`.

1. The last line calls `og` with operands `'print returned'` and `print(2)`; the second operand is `f_B(2)`.
2. In `f_B(2)`: `value = print(x)` calls `f_A(2)`.
3. In `f_A(2)`: `value = print(x)` calls the built-in: displays `2`, `value` is `None`. The `if` is skipped; returns `2`.
4. Back in `f_B`: `value` is `2`, which is not `None`, so it returns `print('What?')`, i.e. `f_A('What?')`.
5. In `f_A('What?')`: the built-in displays `What?`; returns `'What?'`. So `f_B(2)` returns `'What?'`.
6. `og('print returned', 'What?')` displays `print returned What?`.

Displayed:

```
2
What?
print returned What?
```

The lesson is the one from the environments section: inside each `f`, the name `print` means whatever is bound in the frame where that `f` was defined.

E2 (published answer for (b): `n % 2 != n // 10 % 2`; (a): `group_size == k`).

1. Say what the names mean. The loop reads digits from the right. `group_size` is the number of digits read so far in the current group. `count` is the answer so far.
2. (b) When does a group end? When the current last digit and the next digit have different parity: `n % 2 != n // 10 % 2`. Then `group_size` is reset to `0`.
3. (a) Each group must be counted *once*. If we counted whenever `group_size >= k`, a group of 4 digits with `k = 3` would be counted twice (at sizes 3 and 4). Counting exactly when `group_size == k` counts every group that reaches size `k` one time.
4. At the leftmost digit, `n // 10` is `0`, which is even; whether or not the reset happens no longer matters because the loop ends.

Check on `15732429999` with `k = 4`: `9999` reaches size 4 (count 1); `242` reaches only 3; `1573` reaches 4 (count 2).